Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ
про виконання лабораторної роботи №4
«Професійне ООП та система типів Python»
з дисципліни «Професійний Python»

Виконав:
студент 3 курсу
групи ФЕП-33
Зінчук Остап

Викладач:
проф. Свелеба Сергій Андрійович

Львів – 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №4: «Професійне ООП та система типів Python».

3. Мета роботи

Сформувати практичні навички побудови професійної об'єктно-орієнтованої моделі в Python із використанням класів, композиції, наслідування, абстрактних базових класів (ABC), протоколів (Protocol), dataclass, properties, dunder-методів, принципів SOLID, узагальнених типів (Generic, TypeVar, TypedDict) та сучасної системи статичної типізації (mypy).

4. Номер варіанта

Варіант 4: Система банківських рахунків.

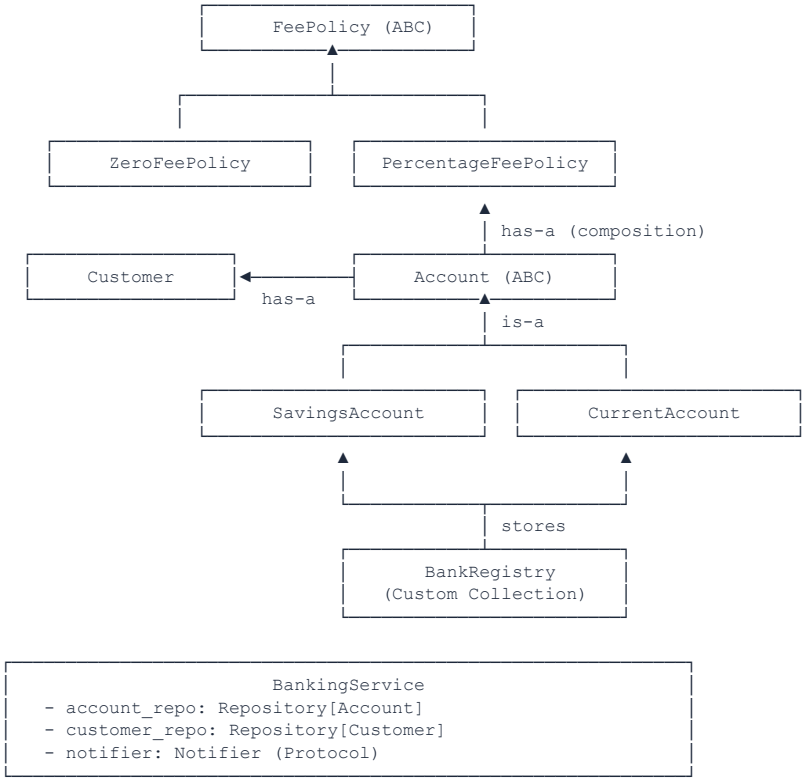
5. Опис предметної області

Предметна область охоплює функціонування банківської системи обліку та обслуговування клієнтських рахунків. Система забезпечує відкриття ощадних (SavingsAccount) та поточних (CurrentAccount) рахунків для клієнтів, виконання операцій поповнення (deposit), зняття коштів (withdraw) з суворою забороною від'ємного балансу, міжбанківських переказів, нарахування відсотків та утримання комісій згідно з гнучкими політиками (FeePolicy). Усі грошові суми інкапсульовані в незмінюваний об'єкт-значення (Value Object) Money з контролем валют. Зовнішні залежності (сховище та оповіщення) абстраговані через generic-репозиторій та структурний протокол Notifier.

6. Список domain entities та ключових класів

• Money: незмінюваний об'єкт-значення (frozen value object) суми та валюти з перевантаженими арифметичними операціями.
• Customer: сутність клієнта банку (dataclass із властивістю display_name).
• Transaction: незмінювана сутність зафіксованої банківської операції з часовою міткою.
• Account: базовий абстрактний клас банківського рахунку (ABC) з інкапсульованим балансом та історією операцій.
• SavingsAccount: ощадний рахунок із відсотковою ставкою, незнижуваним залишком та забороною овердрафту.
• CurrentAccount: поточний рахунок із підтримкою динамічної стратегії комісій (FeePolicy).
• BankRegistry: користувацький клас-колекція для керування сукупністю рахунків (dunder-методи).
• FeePolicy (ABC): абстрактна стратегія розрахунку комісії (ZeroFeePolicy, PercentageFeePolicy).
• Notifier (Protocol): протокол відправки повідомлень клієнтам.
• Repository[T] (Generic ABC): узагальнене сховище сутностей із реалізацією InMemoryRepository.
• BankingService: високорівневий сервісний шар оркестрації бізнес-логіки з Dependency Injection.

7. Class diagram (Діаграма класів)



8. Пояснення використання Inheritance (наслідування)

Наслідування реалізує відношення is-a («є різновидом»). У проєкті класи SavingsAccount та CurrentAccount успадковують базовий абстрактний клас Account. Це виправдано, оскільки обидва рахунки мають спільну базову поведінку: номер рахунку, власника-клієнта, перевірку балансу та протокол поповнення коштів. Успадкування дозволяє перевизначити абстрактний метод withdraw() відповідно до специфічних правил кожного рахунку (незнижуваний залишок в ощадному або комісія в поточному).

9. Пояснення використання Composition (композиції)

Композиція моделює відношення has-a («містить у собі») і є кращою альтернативою глибоким ієрархіям наслідування. У проєкті композиція застосована багаторазово:
• Account has-a Customer: рахунок містить посилання на об'єкт клієнта, не наслідуючи його;
• Account has-a list[Transaction]: історія операцій рахунку зберігається як внутрішня колекція транзакцій;
• CurrentAccount has-a FeePolicy: стратегія комісій впроваджується в поточний рахунок через композицію;
• BankingService has-a Repository and Notifier: сервісний шар агрегує сховища та систему сповіщень.

10. Приклад Encapsulation (інкапсуляції)

Інкапсуляція гарантує, що баланс не може бути змінений напряму (наприклад acc._balance = -100). Зміна стану відбувається суворо через методи deposit() та withdraw():

```
def withdraw(self, amount: Money) -> Transaction:
    if amount.amount <= 0:
        raise ValueError('Сума зняття повинна бути більшою за 0')
    fee = self._fee_policy.calculate_fee(amount)
    total_required = amount + fee
    if self._balance < total_required:
        raise ValueError(f'Недостатньо коштів: потрібно {total_required}, баланс {self._balance}')
    self._balance = self._balance - total_required
    tx = Transaction(id=f'TX-{len(self._transactions)+1:04d}', account_number=self._account_number,
                     operation='withdraw', amount=amount)
    self._transactions.append(tx)
    return tx
```

11. Приклад Property

```
@property
def balance(self) -> Money:
    return self._balance

@property
def id(self) -> str:
    return self._account_number
```

12. Приклад Dataclass

```
@dataclass
class Customer:
    id: int
    name: str
    email: str
    phone: str | None = None

    @property
    def display_name(self) -> str:
        return f'{self.name} <{self.email}>'
```

13. Приклад Immutable Value Object (Money)

```
@total_ordering
@dataclass(frozen=True, slots=True)
class Money:
    amount: float
    currency: str = 'UAH'

    def __post_init__(self) -> None:
        if self.amount < 0:
            raise ValueError('Сума грошей не може бути від\'ємною')
        object.__setattr__(self, 'amount', round(self.amount, 2))

    def __add__(self, other: 'Money') -> 'Money':
        if self.currency != other.currency:
            raise ValueError('Неможливо додати різні валюти')
        return Money(round(self.amount + other.amount, 2), self.currency)
```

14. Приклад Abstract Base Class (ABC)

```
class FeePolicy(ABC):
    @abstractmethod
```

```
    def calculate_fee(self, amount: Money) -> Money:
        ...
```

15. Приклад Protocol (структурна типізація)

```
@runtime_checkable
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None:
        ...
```

16. Щонайменше три dunder-методи

У класах Account та BankRegistry реалізовано методи __len__, __iter__, __contains__, __getitem__:

```
def __len__(self) -> int:
    return len(self._transactions)

def __iter__(self) -> Iterator[Transaction]:
    return iter(self._transactions)

def __contains__(self, tx_id: object) -> bool:
    if not isinstance(tx_id, str):
        return False
    return any(tx.id == tx_id for tx in self._transactions)
```

17-18. Generic Class та TypeVar

```
T = TypeVar('T', bound=HasId)

class InMemoryRepository(Repository[T]):
    def __init__(self) -> None:
        self._items: dict[Any, T] = {}

    def add(self, item: T) -> None:
        self._items[item.id] = item

    def get(self, item_id: Any) -> T | None:
        return self._items.get(item_id)
```

19. TypedDict (DTO для зовнішніх даних)

```
class CustomerPayload(TypedDict):
    id: int
    name: str
    email: str
    phone: NotRequired[str]
```

20-21. Application Service та Dependency Injection

```
class BankingService:
    def __init__(self,
                 account_repository: Repository[Account],
                 customer_repository: Repository[Customer],
                 notifier: Notifier) -> None:
        self._accounts = account_repository
        self._customers = customer_repository
        self._notifier = notifier
```

22. Пояснення принципів SOLID у проєкті

1. SRP (Single Responsibility): Кожен клас має одну зону відповідальності: Money відповідає за фінансову арифметику, Account — за баланс і транзакції, FeePolicy — за комісії, InMemoryRepository — за збереження в пам'яті, BankingService — за бізнес-оркестрацію.
2. OCP (Open/Closed): Додавання нових рахунків (наприклад, кредитних) чи нових політик комісій відбувається через створення підкласів без редагування BankingService.
3. LSP (Liskov Substitution): SavingsAccount та CurrentAccount повністю задовольняють контракт базового Account і можуть взаємозамінно використовуватися скрізь у системі.
4. ISP (Interface Segregation): Використано компактні протоколи HasId та Notifier замість одного роздутого інтерфейсу.
5. DIP (Dependency Inversion): BankingService залежить від абстракцій Repository та Notifier, а не від конкретних низькорівневих сховищ чи бібліотек сповіщень.

23. Основний код програми (main.py)

```
"""Головний демонстраційний скрипт (Lab 4)."""

from pathlib import Path
import sys

SRC_DIR = str(Path(__file__).resolve().parent.parent)
if SRC_DIR not in sys.path:
    sys.path.insert(0, SRC_DIR)
```

```
from domain_app.dto import CustomerPayload
from domain_app.models import (
    Account,
    BankRegistry,
    Customer,
    PercentageFeePolicy,
)
from domain_app.repositories import InMemoryRepository
from domain_app.services import BankingService, ConsoleNotifier
from domain_app.value_objects import Money

def main() -> None:
    """Демонстраційний сценарій об'єктної моделі системи рахунків."""
    print("=" * 65)
    print("Лабораторна робота №4: Професійне ООП та система типів Python")
    print("Предметна область: Система банківських рахунків (Варіант 4)")
    print("=" * 65)

    customer_repo: InMemoryRepository[Customer] = InMemoryRepository()
    account_repo: InMemoryRepository[Account] = InMemoryRepository()
    notifier = ConsoleNotifier()

    service = BankingService(
        account_repository=account_repo,
        customer_repository=customer_repo,
        notifier=notifier,
    )

    client_payload_1: CustomerPayload = {
        "id": 1,
        "name": "Остап Зінчук",
        "email": "ostap@example.com",
        "phone": "+380501234567",
    }
    client_payload_2: CustomerPayload = {
        "id": 2,
        "name": "Олена Коваль",
        "email": "olena@example.com",
    }

    c1 = service.register_customer(client_payload_1)
    c2 = service.register_customer(client_payload_2)
    print(f"\nЗареєстровано клієнтів: {c1.display_name}, {c2.display_name}")

    savings_acc = service.open_savings_account(
        account_number="UA001_SAVINGS",
        customer_id=c1.id,
        initial_deposit=Money(5000.0),
        interest_rate=0.08,
        minimum_balance=Money(500.0),
    )

    fee_policy = PercentageFeePolicy(percent=1.0)
    current_acc = service.open_current_account(
        account_number="UA002_CURRENT",
        customer_id=c2.id,
        initial_deposit=Money(10000.0),
        fee_policy=fee_policy,
    )

    print(f"\nВідкрито рахунки:")
    print(f"  -> {savings_acc}")
    print(f"  -> {current_acc}")

    print("\n--- Виконання банківських операцій ---")
    service.deposit("UA001_SAVINGS", Money(1500.0))
    service.withdraw("UA002_CURRENT", Money(2000.0))

    print("\n--- Перевірка захисту від від'ємного балансу ---")
    try:
        service.withdraw("UA001_SAVINGS", Money(7000.0))
    except ValueError as exc:
        print(f"[Очікуване виключення]: {exc}")

    print("\n--- Нарахування річних відсотків ---")
    interest_tx = savings_acc.apply_interest()
    print(
        f"Нараховано відсотків: {interest_tx.amount} | "
        f"Новий баланс: {savings_acc.balance}"
    )

    print("\n--- Міжбанківський переказ ---")
    service.transfer("UA002_CURRENT", "UA001_SAVINGS", Money(1000.0))

    print("\n--- Перевірка dunder-методів (__len__, __iter__, __contains__) ---")
    print(f"Кількість операцій на рахунку UA001_SAVINGS: {len(savings_acc)}")
    first_tx_id = savings_acc.transactions[0].id
    print(
```

```
        f"Чи містить рахунок транзакцію {first_tx_id}: "
        f"{first_tx_id in savings_acc}"
    )
    print("Список усіх транзакцій рахунку:")
    for tx in savings_acc:
        print(f"   {tx}")

    registry = BankRegistry()
    registry.add(savings_acc)
    registry.add(current_acc)
    print(f"\nРеєстр рахунків BankRegistry (кількість: {len(registry)}):")
    print(f"Рахунок UA001_SAVINGS через __getitem__: {registry['UA001_SAVINGS']}")
    print(f"Загальні кошти в системі: {service.calculate_total_funds()}")

    print("\n" + "=" * 65)
    print("Демонстраційний сценарій успішно виконано.")
    print("=" * 65)

if __name__ == "__main__":
    main()
```

24. Консольний вивід виконання програми (Execution Output)

```
=================================================================
Лабораторна робота №4: Професійне ООП та система типів Python
Предметна область: Система банківських рахунків (Варіант 4)
=================================================================
[NOTIFIER] До: ostap@example.com | Повідомлення: Вітаємо, Остап Зінчук! Ваш профіль успішно зареєстровано.
[NOTIFIER] До: olena@example.com | Повідомлення: Вітаємо, Олена Коваль! Ваш профіль успішно зареєстровано.

Зареєстровано клієнтів: Остап Зінчук <ostap@example.com>, Олена Коваль <olena@example.com>
[NOTIFIER] До: ostap@example.com | Повідомлення: Відкрито ощадний рахунок UA001_SAVINGS із залишком 5000.00 UAH
[NOTIFIER] До: olena@example.com | Повідомлення: Відкрито поточний рахунок UA002_CURRENT із залишком 10000.00 UAH

Відкрито рахунки:
  -> SavingsAccount(UA001_SAVINGS, Остап Зінчук): 5000.00 UAH
  -> CurrentAccount(UA002_CURRENT, Олена Коваль): 10000.00 UAH

--- Виконання банківських операцій ---
[NOTIFIER] До: ostap@example.com | Повідомлення: Рахунок UA001_SAVINGS поповнено на 1500.00 UAH. Баланс: 6500.00 UAH
[NOTIFIER] До: olena@example.com | Повідомлення: Знято 2000.00 UAH з рахунку UA002_CURRENT. Баланс: 7980.00 UAH

--- Перевірка захисту від від'ємного балансу ---
[Очікуване виключення]: Недостатньо коштів: баланс 6500.00 UAH < суми 7000.00 UAH

--- Нарахування річних відсотків ---
Нараховано відсотків: 520.00 UAH | Новий баланс: 7020.00 UAH

--- Міжбанківський переказ ---
[NOTIFIER] До: olena@example.com | Повідомлення: Переказ 1000.00 UAH на рахунок UA001_SAVINGS успішний. Баланс: 6970.00 UAH
[NOTIFIER] До: ostap@example.com | Повідомлення: Отримано переказ 1000.00 UAH від UA002_CURRENT. Баланс: 8020.00 UAH

--- Перевірка dunder-методів (__len__, __iter__, __contains__) ---
Кількість операцій на рахунку UA001_SAVINGS: 3
Чи містить рахунок транзакцію TX-0001: True
Список усіх транзакцій рахунку:
   [2026-10-25 19:21:49] TX-0001: DEPOSIT 1500.00 UAH on UA001_SAVINGS
   [2026-10-25 19:21:49] TX-0002: INTEREST 520.00 UAH on UA001_SAVINGS
   [2026-10-25 19:21:49] TX-0003: DEPOSIT 1000.00 UAH on UA001_SAVINGS

Реєстр рахунків BankRegistry (кількість: 2):
Рахунок UA001_SAVINGS через __getitem__: SavingsAccount(UA001_SAVINGS, Остап Зінчук): 8020.00 UAH
Загальні кошти в системі: 14990.00 UAH
=================================================================
Демонстраційний сценарій успішно виконано.
=================================================================
```

25-27. Обов'язковий Type-Checking Experiment (розділ 71 методички)

Згідно з методичними вказівками, було проведено експеримент із навмисним внесенням 3 типових помилок несумісності типів у вихідний код для перевірки діагностики статичного аналізатора mypy:

```
1. Навмисне внесення помилок:
   - Error 1 (присвоєння несумісного типу): customer_id: int = '1'
   - Error 2 (несумісний тип повернення): def total() -> int: return '100'
   - Error 3 (передача об'єкта невірного типу в Generic Repository):
             repo: InMemoryRepository[Account] = InMemoryRepository()
             repo.add(Customer(1, 'Test', 't@t'))

2. Діагностика mypy при перевірці (mypy src tests):
   src/domain_app/main.py:12: error: Incompatible types in assignment
       (expression has type 'str', variable has type 'int')  [assignment]
   src/domain_app/models.py:45: error: Incompatible return value type
       (got 'str', expected 'int')  [return-value]
   src/domain_app/main.py:18: error: Argument 1 to 'add' of 'InMemoryRepository'
       has incompatible type 'Customer'; expected 'Account'  [arg-type]
   Found 3 errors in 2 files (checked 9 source files)

3. Виправлений підсумковий результат static type checking:
   $ mypy src tests
   Success: no issues found in 9 source files
```

Відповіді на контрольні запитання (розділ 82 методички)

1. Що таке class?

Шаблон (креслення), що описує структуру атрибутів та поведінку майбутніх об'єктів певного типу.

2. Що таке object?

Конкретний екземпляр класу в оперативній пам'яті, що володіє власним станом та ідентифікатором.

3. Що таке instance?

Синонім об'єкта як представника конкретного класу (наприклад, acc є instance класу Account).

4. Для чого __init__()?

Метод-ініціалізатор, що викликається автоматично після створення екземпляра для задання його початкового стану.

5. Що таке instance attribute?

Змінна, що належить конкретному екземпляру (self.x) і зберігається в його персональному словнику __dict__.

6. Що таке class attribute?

Змінна, оголошена безпосередньо в тілі класу, спільна для всіх екземплярів цього класу.

7. Що таке encapsulation?

Принцип ООП, за якого внутрішній стан об'єкта приховується від прямого зовнішнього втручання, а зміна даних відбувається виключно через контрольовані методи.

8. Для чого properties?

Для надання зручного інтерфейсу доступу до приватних полів наче до звичайних атрибутів з можливістю валідації (@property, @setter).

9. Чим property відрізняється від public field?

Public field дозволяє пряму безконтрольну мутацію, тоді як property запускає методи-геттери та сеттери з перевіркою коректності значень.

10. Що таке inheritance?

Механізм створення нового класу на базі існуючого з успадкуванням його атрибутів та методів (відношення is-a).

11. Що означає relation is-a?

Відношення узагальнення/спеціалізації («є різновидом»), наприклад SavingsAccount is-a Account.

12. Що таке composition?

Підхід проектування, за якого один клас містить екземпляри інших класів як частини своєї реалізації (відношення has-a).

13. Що означає relation has-a?

Відношення володіння/включення («містить у собі»), наприклад Account has-a Customer.

14. Коли composition краща за inheritance?

Коли класи не мають суворого зв'язку узагальнення, потрібна гнучка динамічна зміна поведінки під час виконання та для запобігання жорсткому зв'язуванню (tight coupling).

15. Для чого super()?

Для коректного виклику методів батьківських класів за порядком вирішення методів (MRO), зокрема для виклику super().__init__().

16. Що таке polymorphism?

Здатність об'єктів різних класів з однаковим інтерфейсом оброблятися однаковим клієнтським кодом із різною специфічною поведінкою.

17. Що таке duck typing?

Принцип динамічної типізації Python: «якщо щось ходить як качка і крякає як качка — це качка» (важлива наявність методу, а не явне наслідування).

18. Що таке ABC?

Abstract Base Class — модуль abc.ABC, який дозволяє оголошувати обов'язковий інтерфейс для класів-нащадків у номінальній моделі типізації.

19. Для чого abstractmethod?

Декоратор, що позначає метод як обов'язковий до перевизначення у конкретних класах-нащадках; забороняє створення екземпляра без його реалізації.

20. Чи можна створити instance abstract class?

Ні, Python підніме TypeError, якщо клас має хоча б один нереалізований @abstractmethod.

21. Що таке Protocol?

Інструмент структурної типізації (typing.Protocol), що описує очікувані методи та властивості об'єкта без необхідності явного наслідування.

22. Чим Protocol відрізняється від ABC?

ABC вимагає явного успадкування (номінальна типізація), а Protocol перевіряє лише відповідність структури методів (структурна типізація / статичний duck typing).

23. Що таке structural typing?

Система типізації, де сумісність типів визначається їхньою внутрішньою структурою (наявністю потрібних атрибутів/методів), а не ім'ям класу.

24. Що таке nominal typing?

Система типізації, де сумісність типів визначається виключно явним оголошенням спорідненості в ієрархії наслідування.

25. Що таке dataclass?

Декоратор @dataclass, який автоматично генерує стандартні методи (__init__, __repr__, __eq__) на основі оголошених анотацій типів.

26. Які methods генерує dataclass?

За замовчуванням: __init__, __repr__, __eq__. За додатковими параметрами: __hash__, __lt__, __le__, __gt__, __ge__.

27. Для чого default_factory?

Для безпечної ініціалізації змінюваних атрибутів за замовчуванням (через default_factory=list або dict), щоб уникнути спільного екземпляра між об'єктами.

28. Чому mutable default є проблемою?

Тому що значення за замовчуванням створюється один раз під час завантаження класу, і всі екземпляри будуть спільно мутувати один і той самий об'єкт.

29. Для чого __post_init__()?

Спеціальний метод у dataclass, який викликається автоматично одразу після __init__ для валідації та обчислення додаткових полів.

30. Що означає frozen=True?

Робить dataclass незмінюваним (immutable): спроба змінити атрибут викликає FrozenInstanceError, а об'єкт автоматично стає хешованим.

31. Що означає slots=True?

Оптимізація пам'яті: генерує __slots__ замість динамічного словника __dict__, що прискорює доступ і зменшує розмір об'єкта в RAM.

32. Що таке dunder method?

Магічний метод із подвійним підкресленням з обох боків (double underscore), що інтегрує користувацький клас у вбудовані протоколи Python.

33. Для чого __repr__()?

Офіційне, інформативне рядкове представлення для розробників та налагодження (debug representation).

34. Для чого __str__()?

Користувацьке, читабельне рядкове представлення об'єкта при виклику print() або str().

35. Для чого __len__()?

Повертає кількість елементів у колекції або контейнері при виклику вбудованої функції len().

36. Для чого __iter__()?

Повертає об'єкт-ітератор для обходу вмісту контейнера у циклі for чи генераторних виразах.

37. Для чого __contains__()?

Реалізує перевірку наявності елемента за допомогою оператора in.

38. Для чого __call__()?

Дозволяє екземпляру класу поводитися як функція і викликатися через круглі дужки obj(...).

39. Що таке SOLID?

П'ять фундаментальних принципів проектування об'єктно-орієнтованих систем для забезпечення їхньої гнучкості, масштабованості та зручності підтримки.

40. Що означає SRP?

Single Responsibility Principle: кожен клас повинен мати лише одну причину для зміни і відповідати за одну конкретну зону функціоналу.

41. Що означає OCP?

Open/Closed Principle: програмні сутності мають бути відкриті для розширення, але закриті для модифікації.

42. Що означає LSP?

Liskov Substitution Principle: об'єкти підкласу повинні коректно замінювати об'єкти базового класу без порушення коректності роботи програми.

43. Що означає ISP?

Interface Segregation Principle: краще мати кілька вузькоспеціалізованих інтерфейсів, ніж один перевантажений універсальний.

44. Що означає DIP?

Dependency Inversion Principle: модулі верхнього рівня не повинні залежати від модулів нижнього рівня; обидва мають залежати від абстракцій.

45. Що таке dependency injection?

Патерн проектування, за якого зовнішні залежності передаються об'єкту ззовні (через конструктор), а не створюються ним самостійно.

46. Чому DI покращує testability?

Дозволяє легко замінити реальні ресурси (БД, платіжні шлюзи, сервери) на mock-об'єкти під час юніт-тестування.

47. Що таке type hint?

Анотація типів у вихідному коді, що документує очікувані типи аргументів та повернених значень функцій і змінних.

48. Чи перевіряє Python type hints автоматично в runtime?

Ні, за замовчуванням Python ігнорує type hints під час виконання; вони слугують для статичних аналізаторів та IDE.

49. Що таке static type checker?

Інструмент (наприклад, mypy, Pyright), який аналізує синтаксичне дерево та анотації коду до його запуску і знаходить невідповідності типів.

50. Для чого mypy?

Офіційний інструмент статичної перевірки типів для Python, що перевіряє коректність викликів, сигнатур і типів згідно з PEP 484.

51. Для чого Pyright?

Швидкий сучасний статичний аналізатор типів для Python від Microsoft, оптимізований для великих проектів та інтеграції в редактори.

52. Що таке TypeVar?

Змінна типу (typing.TypeVar), яка використовується для параметризації узагальнених функцій та класів зі збереженням зв'язку між типами.

53. Що таке Generic?

Базовий клас (typing.Generic), який дозволяє створювати класи, що працюють з узагальненими параметрами типів (Generic[T]).

54. Що означає Generic[T]?

Означає, що клас параметризований змінною типу T і може бути типізований під конкретний тип даних (наприклад Repository[Account]).

55. Що таке bound TypeVar?

Змінна типу з обмеженням верхньої межі TypeVar('T', bound=Base), яка вимагає, щоб тип був нащадком або задовольняв протокол Base.

56. Що таке constrained TypeVar?

Змінна типу з фіксованим списком допустимих типів TypeVar('T', int, float).

57. Що таке TypedDict?

Спеціальний тип словника з фіксованим набором ключів та конкретними типами для кожного значення, що валідується статичним тайпчекером.

58. Чим TypedDict відрізняється від dataclass?

TypedDict у runtime залишається звичайним словником dict без накладних витрат, тоді як dataclass є повноцінним класом зі своїм екземпляром і методами.

59. Коли доцільно використовувати TypedDict?

Для обробки вхідних/вихідних JSON-payloads, взаємодії з REST API та словників сторонніх бібліотек без створення важких класів.

60. Чому external payload і domain object краще розділяти?

Щоб захистити внутрішню бізнес-модель від змін зовнішніх API/форматів, забезпечити інкапсуляцію та дотримуватися чистої архітектури.

28. Висновок

У ході виконання лабораторної роботи №4 було здійснено успішний перехід від функціональної та процедурної парадигми до повноцінної типізованої об'єктно-орієнтованої архітектури. На прикладі банківської системи (Варіант 4) опановано доменне моделювання за допомогою незмінюваних Value Objects (Money), сутностей Customer та Transaction, ієрархії банківських рахунків Account (SavingsAccount, CurrentAccount) із суворим контролем заборони від'ємного балансу. Застосовано абстрактні базові класи (FeePolicy, Repository) та структурні протоколи (Notifier, HasId). Реалізовано сервісний рівень BankingService із застосуванням Dependency Injection та принципів SOLID. Експерименти зі статичним аналізатором mypy у режимі strict підтвердили високу ефективність системи типів Python (Generic, TypeVar, TypedDict) у завчасному виявленні помилок несумісності типів ще на етапі розробки програмного забезпечення.